

Real-world (persist and read back real data) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `sqlite_persistence_practice.py`: this is the Build Task's run history in miniature — save every request's input and output, then read one back by its ID. Doing it first on a one-line fake chat keeps the SQL the only new thing. **If not:** the Build Task's `runs` table, its `GET /runs/{id}` route, and its 404 case would all be first attempts, mixed in with a multi-agent pipeline.

Every version below creates `runs.db` in `practice/` the first time it runs. Delete that file between tries if you want a clean slate. Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# practice/sqlite_persistence_practice.py – Real-world section
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        created_at TEXT
    )
""")

class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
```

```
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    conn.execute(
        "INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
        (request.message, reply, now),
    )
    conn.commit()
    return {"reply": reply}

@app.get("/runs")
def get_runs():
    rows = conn.execute("SELECT * FROM runs").fetchall()
    return rows
```

Expected output (`POST /chat` with `{"message": "hi"}` , then `GET /runs`):

```
POST /chat -> {"reply": "You said: hi"}
GET /runs -> [[1, "hi", "You said: hi", "2026-01-01T00:00:00.000000+00:00"]]
```

This saves and reads back real data — the core requirement — but `GET /runs` returns unlabeled tuples, which is valid JSON but not something a real client could use without guessing the field order.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — `sqlite3.Row`, real dicts back out

Story: a client reading `row[2]` has to know the table's column order by heart, and breaks the day a column is added. Named fields fix that. **If not:** the Build Task's `GET /runs/{id}` would hand back a bare list, and no response model could check it.

```
# practice/sqlite_persistence_practice.py – Real-world section
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
# how: every row now behaves like a dict – row["input"] works
conn.row_factory = sqlite3.Row
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        created_at TEXT
    )
""")

class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    # why: ? placeholders – values are never pasted into the SQL text
    conn.execute(
        "INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
        (request.message, reply, now),
    )
    conn.commit()
    return {"reply": reply}
```

```
@app.get("/runs")
def get_runs():
    rows = conn.execute("SELECT * FROM runs").fetchall()
    result = []
    for row in rows:
        # how: dict(row) turns a sqlite3.Row into a real dict
        result.append(dict(row))
    return result
```

Expected output (GET /runs ; shown wrapped onto 2 lines just to fit the page — really one line of output):

```
[{"id":1,"input":"hi","output":"You said: hi",
  "created_at":"2026-01-01T00:00:00.000000+00:00"}]
```

Approach 2 — a small helper file instead of SQL inside the routes

Story: SQL scattered through route functions is hard to find and hard to change. A helper file keeps all database code in one place, and the routes just call named functions. **If not:** the Build Task's `db/database.py` split would be a new idea instead of one you've already practiced.

```
# practice/runs_db.py
import sqlite3

def get_connection():
    conn = sqlite3.connect("runs.db", check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute("""
        CREATE TABLE IF NOT EXISTS runs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            input TEXT,
            output TEXT,
            created_at TEXT
        )
    """)
    return conn

def insert_run(conn, input_text, output_text, created_at):
    conn.execute(
        "INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
        (input_text, output_text, created_at),
    )
    conn.commit()
```

```
def fetch_all_runs(conn):
    rows = conn.execute("SELECT * FROM runs").fetchall()
    result = []
    for row in rows:
        result.append(dict(row))
    return result
```

```
# practice/sqlite_persistence_practice.py – Real-world section
from datetime import datetime, timezone
from fastapi import FastAPI
from pydantic import BaseModel
from runs_db import get_connection, insert_run, fetch_all_runs

app = FastAPI()
conn = get_connection()

class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    insert_run(conn, request.message, reply, now)
    return {"reply": reply}

@app.get("/runs")
def get_runs():
    return fetch_all_runs(conn)
```

Expected output: identical to Approach 1 — this only reorganizes the code.

Approach 3 — a typed `response_model`, and `GET /runs/{id}` with a 404

Story: real clients rarely want the whole table — they want "what happened in run 7?". A lookup by ID has to handle "there is no run 7" as a clear 404, not a crash. **If not:** the Build Task's `GET /runs/{id}` and its "wrong ID" case would be written for the first time there.

```
# practice/sqlite_persistence_practice.py – Real-world section
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
```

```

conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.row_factory = sqlite3.Row
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        created_at TEXT
    )
""")

class ChatRequest(BaseModel):
    message: str

# why: one saved row's shape, checked on the way out –
# same idea as ChatResponse in chat_route
class RunRecord(BaseModel):
    id: int
    input: str
    output: str
    created_at: str

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    conn.execute(
        "INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
        (request.message, reply, now),
    )
    conn.commit()
    return {"reply": reply}

@app.get("/runs", response_model=list[RunRecord])
def get_runs():
    rows = conn.execute("SELECT * FROM runs").fetchall()
    result = []
    for row in rows:
        result.append(dict(row))
    return result

@app.get("/runs/{run_id}", response_model=RunRecord)
def get_run(run_id: int):
    # how: (run_id,) – even one value must be passed as a tuple
    sql = "SELECT * FROM runs WHERE id = ?"
    row = conn.execute(sql, (run_id,)).fetchone()
    # when: fetchone() returns None when no row matches

```

```

if row is None:
    raise HTTPException(status_code=404, detail="run not found")
return dict(row)

```

Expected output for `GET /runs/1` (existing row; shown wrapped onto 2 lines just to fit the page — really one line of output):

```

200 {"id":1,"input":"hi","output":"You said: hi",
    "created_at":"2026-01-01T00:00:00.000000+00:00"}

```

Expected output for `GET /runs/999` (no such row):

```

404 {"detail":"run not found"}

```

`GET /runs/abc` gives a 422 on its own, because `run_id` must be an `int` — FastAPI checks path values too.

Difference from Basic: Approach 1 adds `conn.row_factory = sqlite3.Row` and converts each row with `dict(row)`, so `GET /runs` returns field-named JSON instead of positional tuples. Approach 2 moves the SQL into a dedicated `runs_db.py`, keeping the route file focused on routing — the same move `chat_route`'s Approach 2 made for its models. Approach 3 adds a typed `RunRecord` response model and a single-row lookup that handles "not found" as a clean 404.

Which one should you actually write? Approach 1 (`row_factory` + `dict(row)`) is the minimum any SQLite-backed route needs — never return a raw `sqlite3.Row` or tuple from a route. Add Approach 3's `RunRecord` and `GET /runs/{id}` as soon as a client needs to look up one specific run, which the Build Task requires. Use Approach 2's helper file once there's more than one or two SQL statements — the Build Task's `db/database.py` is exactly this file.